
Please note: Some of the questions in this former practice exam may no longer perfectly align with the AP exam. Even though these questions do not fully represent the 2020 exam, teachers indicate that imperfectly aligned questions still provide instructional value. Teachers can consult the Question Bank to determine the degree to which these questions align to the 2020 Exam.

This exam may not be posted on school or personal websites, nor electronically redistributed for any reason. This exam is provided by the College Board for AP Exam preparation. Teachers are permitted to download the materials and make copies to use with their students in a classroom setting only. To maintain the security of this exam, teachers should collect all materials after their administration and keep them in a secure location.

Further distribution of these materials outside of the secure College Board site disadvantages teachers who rely on uncirculated questions for classroom testing. Any additional distribution is in violation of the College Board's copyright policies and may result in the termination of Practice Exam access for your school as well as the removal of access to other online services such as the AP Teacher Community and Online Score Reports.

Collect an answer sheet from each student. Check that each answer sheet has an AP number label and an AP Exam label. After all answer sheets have been collected, say:

Now you must seal your exam booklet using the white seals you set aside earlier. Remove the white seals from the backing and press one on each area of your exam booklet cover marked “PLACE SEAL HERE.” Fold each seal over the back cover. When you have finished, place the booklet on your desk, face up. . . .

I will now collect your Section I booklet.

Collect a Section I booklet from each student. Check that each student has signed the front cover of the sealed Section I booklet.

There is a 10-minute break between Sections I and II. When all Section I materials have been collected and accounted for and you are ready for the break, say:

Please listen carefully to these instructions before we take a 10-minute break. All items you placed under your chair at the beginning of this exam must stay there, and you are not permitted to open or access them in any way. Leave your shrinkwrapped Section II packet on your desk during the break. You are not allowed to consult teachers, other students, or textbooks during the break. You may not make phone calls, send text messages, check email, use a social networking site, or access any electronic or communication device. Remember, you may never discuss the multiple-choice questions at any time in any form with anyone, including your teacher and other students. If you disclose these questions through any means, your AP Exam score will be canceled. Are there any questions? . . .



You may begin your break. Testing will resume at _____.

SECTION II: Free Response

After the break, say:

May I have everyone’s attention? Place your Student Pack on your desk. . . .

You may now remove the shrinkwrap from the Section II packet, but do not open the exam booklet until you are told to do so. . . .

Read the bulleted statements on the front cover of the exam booklet. Look up when you have finished. . . .

Now place an AP number label on the shaded box. If you don’t have any AP number labels, write your AP number in the box. Look up when you have finished. . . .

Read the last statement. . . .

Using your pen, print the first, middle and last initials of your legal name in the boxes and print today’s date where indicated. This constitutes your signature and your agreement to the statements on the front cover. . . .

Turn to the back cover and complete Item 1 under “Important Identification Information.” Print the first two letters of your last name and the first letter of your first name in the boxes. Look up when you have finished. . . .

In Item 2, print your date of birth in the boxes. . . .

In Item 3, write the school code you printed on the front of your Student Pack in the boxes. . . .

Read Item 4. . . .

Are there any questions? . . .

I need to collect the Student Pack from anyone who will be taking another AP Exam. You may keep it only if you are not taking any other AP Exams this year. If you have no other AP Exams to take, place your Student Pack under your chair now. . . .

While Student Packs are being collected, read the information on the back cover of the exam booklet. Do not open the booklet until you are told to do so. Look up when you have finished. . . .

Collect the Student Packs. Then say:

Are there any questions? . . .

You have 1 hour and 45 minutes for Section II. You must write your answers in the exam booklet using a No. 2 pencil. You are responsible for pacing yourself, and may proceed freely from one question to the next. If you need more paper during the exam, raise your hand. At the top of each sheet of paper you use, be sure to write only your AP number and the number of the question you are working on. Do not write your name. The Java Quick Reference is located at the back of the booklet. Are there any questions? . . .

You may begin.



Note Start Time here _____. Note Stop Time here _____. After 1 hour and 35 minutes, say:

There are 10 minutes remaining.

After 10 minutes, say:

Stop working and close your exam booklet. Place it on your desk, face up. . . .

If any students used extra paper for the free-response section, have those students staple the extra sheet(s) to the first page corresponding to that question in their exam booklets. Complete an Incident Report and include any exam booklets with extra sheets of paper in an Incident Report return envelope (see page 57 of the *AP Coordinator’s Manual* for details). Then say:

Remain in your seat, without talking, while the exam materials are collected. . . .

Collect a Section II booklet from each student. Check for the following:

- Exam booklet front cover: The student placed an AP number label on the shaded box, and printed his or her initials and today's date.
- Exam booklet back cover: The student completed the "Important Identification Information" area.

When all exam materials have been collected and accounted for, return to students any electronic devices you may have collected before the start of the exam.

If you are giving the regularly scheduled exam, say:

You may not discuss or share these specific free-response questions with anyone unless they are released on the College Board website in about two days. Your AP Exam score results will be available online in July.

If you are giving the alternate exam for late testing, say:

None of the questions in this exam may ever be discussed or shared in any way at any time. Your AP Exam score results will be available online in July.

If any students completed the AP number card at the beginning of this exam, say:

Please remember to take your AP number card with you. You will need the information on this card to view your scores and order AP score reporting services online.

Then say:

You are now dismissed.

All exam materials must be placed in secure storage until they are returned to the AP Program after your school's last administration. Before storing materials, check the "School Use Only" section on page 1 of the answer sheet and:

- Fill in the appropriate section number circle in order to access a separate AP Instructional Planning Report (for regularly scheduled exams only) or subject score roster at the class section or teacher level. See "Post-Exam Activities" in the *2014-15 AP Coordinator's Manual*.
- Check your list of students who are eligible for fee reductions and fill in the appropriate circle on their registration answer sheets.

Be sure to give the completed seating chart to the AP Coordinator. Schools must retain seating charts for at least six months (unless the state or district requires that they be retained for a longer period of time). Schools should not return any seating charts in their exam shipments unless they are required as part of an Incident Report.

AP[®] Computer Science A Exam

SECTION II: Free Response

2015

DO NOT OPEN THIS BOOKLET UNTIL YOU ARE TOLD TO DO SO.

At a Glance

Total Time

1 hour, 45 minutes

Number of Questions

4

Percent of Total Score

50%

Writing Instrument

Pencil

Electronic Device

None allowed

Weight

The questions are weighted equally.

IMPORTANT Identification Information

PLEASE PRINT WITH PEN:

1. First two letters of your last name

First letter of your first name

2. Date of birth

Month Day Year

3. Six-digit school code

4. Unless I check the box below, I grant the College Board the unlimited right to use, reproduce, and publish my free-response materials, both written and oral, for educational research and instructional purposes. My name and the name of my school will not be used in any way in connection with my free-response materials. I understand that I am free to mark "No" with no effect on my score or its reporting.

No, I do not grant the College Board these rights. ☐

Instructions

The questions for Section II are printed in this booklet. You may use any of the blank pages to organize your answers and for scratch work, but you must write your answers on the pages that correspond with the questions.

The Java Quick Reference is located at the back of this booklet.

Write your answer to each question in the space provided for the question in this booklet. Some questions require you to write program segments, and these must be written in Java. Show all your work. Credit for partial solutions will be given. Write clearly and legibly. Cross out any errors you make. Erased or crossed-out work will not be scored.

Manage your time carefully. Do not spend too much time on any one question. You may proceed freely from one question to the next. You may review your responses if you finish before the end of the exam is announced.

COMPUTER SCIENCE A

SECTION II

Time—1 hour and 45 minutes

Number of questions—4

Percent of total score—50

Directions: SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Notes:

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

1. This question involves reasoning about one-dimensional and two-dimensional arrays of integers. You will write three static methods, all of which are in a single enclosing class, named `DiverseArray` (not shown). The first method returns the sum of the values of a one-dimensional array; the second method returns an array that represents the sums of the rows of a two-dimensional array; and the third method analyzes row sums.
- (a) Write a static method `arraySum` that calculates and returns the sum of the entries in a specified one-dimensional array. The following example shows an array `arr1` and the value returned by a call to `arraySum`.

<u>arr1</u>					Value returned by <u>arraySum(arr1)</u>
0	1	2	3	4	
1	3	2	7	3	16

WRITE YOUR SOLUTION ON THE NEXT PAGE.

Complete method `arraySum` below.

```
/** Returns the sum of the entries in the one-dimensional array arr.  
 */  
public static int arraySum(int[] arr)
```

Part (b) begins on page 6.

- (b) Write a static method `rowSums` that calculates the sums of each of the rows in a given two-dimensional array and returns these sums in a one-dimensional array. The method has one parameter, a two-dimensional array `arr2D` of `int` values. The array is in row-major order: `arr2D[r][c]` is the entry at row `r` and column `c`. The method returns a one-dimensional array with one entry for each row of `arr2D` such that each entry is the sum of the corresponding row in `arr2D`. As a reminder, each row of a two-dimensional array is a one-dimensional array.

For example, if `mat1` is the array represented by the following table, the call `rowSums(mat1)` returns the array `{16, 32, 28, 20}`.

	<u>mat1</u>				
	0	1	2	3	4
0	1	3	2	7	3
1	10	10	4	6	2
2	5	3	5	9	6
3	7	6	4	2	1

Methods written in this question

```
public static int arraySum(int[] arr)
public static int[] rowSums(int[][] arr2D)
public static boolean isDiverse(int[][] arr2D)
```

WRITE YOUR SOLUTION ON THE NEXT PAGE.

Assume that `arraySum` works as specified, regardless of what you wrote in part (a). You must use `arraySum` appropriately to receive full credit.

Complete method `rowSums` below.

```
/** Returns a one-dimensional array in which the entry at index k is the sum of
 * the entries of row k of the two-dimensional array arr2D.
 */
public static int[] rowSums(int[][] arr2D)
```

Part (c) begins on page 8.

- (c) A two-dimensional array is *diverse* if no two of its rows have entries that sum to the same value. In the following examples, the array `mat1` is diverse because each row sum is different, but the array `mat2` is not diverse because the first and last rows have the same sum.

	<u>mat1</u>					
	0	1	2	3	4	Row sums
0	1	3	2	7	3	16
1	10	10	4	6	2	32
2	5	3	5	9	6	28
3	7	6	4	2	1	20

	<u>mat2</u>					
	0	1	2	3	4	Row sums
0	1	1	5	3	4	14
1	12	7	6	1	9	35
2	8	11	10	2	5	36
3	3	2	3	0	6	14

Write a static method `isDiverse` that determines whether or not a given two-dimensional array is diverse. The method has one parameter: a two-dimensional array `arr2D` of `int` values. The method should return `true` if all the row sums in the given array are unique; otherwise, it should return `false`. In the arrays shown above, the call `isDiverse(mat1)` returns `true` and the call `isDiverse(mat2)` returns `false`.

Methods written in this question

```
public static int arraySum(int[] arr)
public static int[] rowSums(int[][] arr2D)
public static boolean isDiverse(int[][] arr2D)
```

WRITE YOUR SOLUTION ON THE NEXT PAGE.

Assume that `arraySum` and `rowSums` work as specified, regardless of what you wrote in parts (a) and (b). You must use `rowSums` appropriately to receive full credit.

Complete method `isDiverse` below.

```
/** Returns true if all rows in arr2D have different row sums;  
 *      false otherwise.  
 */  
public static boolean isDiverse(int[][] arr2D)
```

2. Consider a guessing game in which a player tries to guess a hidden word. The hidden word contains only capital letters and has a length known to the player. A guess contains only capital letters and has the same length as the hidden word.

After a guess is made, the player is given a hint that is based on a comparison between the hidden word and the guess. Each position in the hint contains a character that corresponds to the letter in the same position in the guess. The following rules determine the characters that appear in the hint.

If the letter in the guess is ...	the corresponding character in the hint is
also in the same position in the hidden word,	the matching letter
also in the hidden word, but in a different position,	" + "
not in the hidden word,	" * "

The `HiddenWord` class will be used to represent the hidden word in the game. The hidden word is passed to the constructor. The class contains a method, `getHint`, that takes a guess and produces a hint.

For example, suppose the variable `puzzle` is declared as follows.

```
HiddenWord puzzle = new HiddenWord("HARPS");
```

The following table shows several guesses and the hints that would be produced.

Call to <code>getHint</code>	String returned
<code>puzzle.getHint("AAAAA")</code>	<code>" +A+++ "</code>
<code>puzzle.getHint("HELLO")</code>	<code>"H**** "</code>
<code>puzzle.getHint("HEART")</code>	<code>"H*++* "</code>
<code>puzzle.getHint("HARMS")</code>	<code>"HAR*S "</code>
<code>puzzle.getHint("HARPS")</code>	<code>"HARPS "</code>

Write the complete `HiddenWord` class, including any necessary instance variables, its constructor, and the method, `getHint`, described above. You may assume that the length of the guess is the same as the length of the hidden word.

ADDITIONAL WORK SPACE

GO ON TO THE NEXT PAGE.

3. A two-dimensional array of integers in which most elements are zero is called a *sparse array*. Because most elements have a value of zero, memory can be saved by storing only the non-zero values along with their row and column indexes. The following complete `SparseArrayEntry` class is used to represent non-zero elements in a sparse array. A `SparseArrayEntry` object cannot be modified after it has been constructed.

```
public class SparseArrayEntry
{
    /** The row index and column index for this entry in the sparse array */
    private int row;
    private int col;

    /** The value of this entry in the sparse array */
    private int value;

    /** Constructs a SparseArrayEntry object that represents a sparse array element
     * with row index r and column index c, containing value v.
     */
    public SparseArrayEntry(int r, int c, int v)
    {
        row = r;
        col = c;
        value = v;
    }

    /** Returns the row index of this sparse array element. */
    public int getRow()
    { return row; }

    /** Returns the column index of this sparse array element. */
    public int getCol()
    { return col; }

    /** Returns the value of this sparse array element. */
    public int getValue()
    { return value; }
}
```

The `SparseArray` class represents a sparse array. It contains a list of `SparseArrayEntry` objects, each of which represents one of the non-zero elements in the array. The entries representing the non-zero elements are stored in the list in no particular order. Each non-zero element is represented by exactly one entry in the list.

```
public class SparseArray
{
    /** The number of rows and columns in the sparse array. */
    private int numRows;
    private int numCols;

    /** The list of entries representing the non-zero elements of the sparse array. Entries are stored in the
     *  list in no particular order. Each non-zero element is represented by exactly one entry in the list.
     */
    private List<SparseArrayEntry> entries;

    /** Constructs an empty SparseArray. */
    public SparseArray()
    { entries = new ArrayList<SparseArrayEntry>(); }

    /** Returns the number of rows in the sparse array. */
    public int getNumRows()
    { return numRows; }

    /** Returns the number of columns in the sparse array. */
    public int getNumCols()
    { return numCols; }

    /** Returns the value of the element at row index row and column index col in the sparse array.
     *  Precondition:  $0 \leq \text{row} < \text{getNumRows}()$ 
     *                   $0 \leq \text{col} < \text{getNumCols}()$ 
     */
    public int getValueAt(int row, int col)
    { /* to be implemented in part (a) */ }

    /** Removes the column col from the sparse array.
     *  Precondition:  $0 \leq \text{col} < \text{getNumCols}()$ 
     */
    public void removeColumn(int col)
    { /* to be implemented in part (b) */ }

    // There may be instance variables, constructors, and methods that are not shown.
}
```

The following table shows an example of a two-dimensional sparse array. Empty cells in the table indicate zero values.

	0	1	2	3	4
0					
1		5			4
2	1				
3		-9			
4					
5					

The sample array can be represented by a `SparseArray` object, `sparse`, with the following instance variable values. The items in `entries` are in no particular order; one possible ordering is shown below.

`numRows: 6`

`numCols: 5`

entries:	row: 1	row: 2	row: 3	row: 1
	col: 4	col: 0	col: 1	col: 1
	value: 4	value: 1	value: -9	value: 5

- (a) Write the `SparseArray` method `getValueAt`. The method returns the value of the sparse array element at a given row and column in the sparse array. If the list `entries` contains an entry with the specified row and column, the value associated with the entry is returned. If there is no entry in `entries` corresponding to the specified row and column, 0 is returned.

In the example above, the call `sparse.getValueAt(3, 1)` would return -9, and `sparse.getValueAt(3, 3)` would return 0.

WRITE YOUR SOLUTION ON THE NEXT PAGE.

Complete method `getValueAt` below.

```
/** Returns the value of the element at row index row and column index col in the sparse array.  
 * Precondition:  $0 \leq \text{row} < \text{getNumRows}()$   
 *  $0 \leq \text{col} < \text{getNumCols}()$   
 */  
public int getValueAt(int row, int col)
```

Part (b) begins on page 16.

(b) Write the `SparseArray` method `removeColumn`. After removing a specified column from a sparse array:

- All entries in the list `entries` with column indexes matching `col` are removed from the list.
- All entries in the list `entries` with column indexes greater than `col` are replaced by entries with column indexes that are decremented by one (moved one column to the left).
- The number of columns in the sparse array is adjusted to reflect the column removed.

The sample object `sparse` from the beginning of the question is repeated for your convenience.

	0	1	2	3	4
0					
1		5			4
2	1				
3		-9			
4					
5					

The shaded entries in `entries`, below, correspond to the shaded column above.

`numRows: 6`

`numCols: 5`

<code>entries:</code>	row: 1 col: 4 value: 4	row: 2 col: 0 value: 1	row: 3 col: 1 value: -9	row: 1 col: 1 value: 5
-----------------------	---	---	--	---

When `sparse` has the state shown above, the call `sparse.removeColumn(1)` could result in `sparse` having the following values in its instance variables (since `entries` is in no particular order, it would be equally valid to reverse the order of its two items). The shaded areas below show the changes.

`numRows: 6`

`numCols: 4`

<code>entries:</code>	row: 1 col: 3 value: 4	row: 2 col: 0 value: 1
-----------------------	---	---

Class information repeated from the beginning of the question

public class SparseArrayEntry

public SparseArrayEntry(int r, int c, int v)

public int getRow()

public int getCol()

public int getValue()

public class SparseArray

private int numRows

private int numCols

private List<SparseArrayEntry> entries

public int getNumRows()

public int getNumCols()

public int getValueAt(int row, int col)

public void removeColumn(int col)

Complete method `removeColumn` below.

```
/** Removes the column col from the sparse array.  
 * Precondition:  $0 \leq \text{col} < \text{getNumCols}()$   
 */  
public void removeColumn(int col)
```

ADDITIONAL WORK SPACE

GO ON TO THE NEXT PAGE.

4. This question involves the design of an interface, writing a class that implements the interface, and writing a method that uses the interface.

- (a) A *number group* represents a group of integers defined in some way. It could be empty, or it could contain one or more integers.

Write an interface named `NumberGroup` that represents a group of integers. The interface should have a single `contains` method that determines if a given integer is in the group. For example, if `group1` is of type `NumberGroup`, and it contains only the two numbers `-5` and `3`, then `group1.contains(-5)` would return `true`, and `group1.contains(2)` would return `false`.

Write the complete `NumberGroup` interface. It must have exactly one method.

Part (b) begins on page 21.

- (b) A *range* represents a number group that contains all (and only) the integers between a minimum value and a maximum value, inclusive.

Write the `Range` class, which is a `NumberGroup`. The `Range` class represents the group of `int` values that range from a given minimum value up through a given maximum value, inclusive. For example, the declaration

```
NumberGroup range1 = new Range(-3, 2);
```

represents the group of integer values -3, -2, -1, 0, 1, 2.

Write the complete `Range` class. Include all necessary instance variables and methods as well as a constructor that takes two `int` parameters. The first parameter represents the minimum value, and the second parameter represents the maximum value of the range. You may assume that the minimum is less than or equal to the maximum.

Part (c) begins on page 22.

- (c) The `MultipleGroups` class (not shown) represents a collection of `NumberGroup` objects and is a `NumberGroup`. The `MultipleGroups` class stores the number groups in the instance variable `groupList` (shown below), which is initialized in the constructor.

```
private List<NumberGroup> groupList;
```

Write the `MultipleGroups` method `contains`. The method takes an integer and returns `true` if and only if the integer is contained in one or more of the number groups in `groupList`.

For example, suppose `multiple1` has been declared as an instance of `MultipleGroups` and consists of the three ranges created by the calls `new Range(5, 8)`, `new Range(10, 12)`, and `new Range(1, 6)`. The following table shows the results of several calls to `contains`.

Call	Result
<code>multiple1.contains(2)</code>	<code>true</code>
<code>multiple1.contains(9)</code>	<code>false</code>
<code>multiple1.contains(6)</code>	<code>true</code>

Complete method `contains` below.

```
/** Returns true if at least one of the number groups in this multiple group contains num;  
 *      false otherwise.  
 */  
public boolean contains(int num)
```

ADDITIONAL WORK SPACE

GO ON TO THE NEXT PAGE.

Java Quick Reference

Accessible Methods from the Java Library That May Be Included on the Exam

class java.lang.Object

- boolean equals(Object other)
- String toString()

class java.lang.Integer

- Integer(int value)
- int intValue()
- Integer.MIN_VALUE // minimum value represented by an int or Integer
- Integer.MAX_VALUE // maximum value represented by an int or Integer

class java.lang.Double

- Double(double value)
- double doubleValue()

class java.lang.String

- int length()
- String substring(int from, int to) // returns the substring beginning at from
// and ending at to-1
- String substring(int from) // returns substring(from, length())
- int indexOf(String str) // returns the index of the first occurrence of str;
// returns -1 if not found
- int compareTo(String other) // returns a value < 0 if this is less than other
// returns a value = 0 if this is equal to other
// returns a value > 0 if this is greater than other

class java.lang.Math

- static int abs(int x)
- static double abs(double x)
- static double pow(double base, double exponent)
- static double sqrt(double x)
- static double random() // returns a double in the range [0.0, 1.0)

interface java.util.List<E>

- int size()
- boolean add(E obj) // appends obj to end of list; returns true
- void add(int index, E obj) // inserts obj at position index (0 ≤ index ≤ size),
// moving elements at position index and higher
// to the right (adds 1 to their indices) and adjusts size
- E get(int index)
- E set(int index, E obj) // replaces the element at position index with obj
// returns the element formerly at the specified position
- E remove(int index) // removes element from position index, moving elements
// at position index + 1 and higher to the left
// (subtracts 1 from their indices) and adjusts size
// returns the element formerly at the specified position

class java.util.ArrayList<E> implements java.util.List<E>